

Informal notes on the Natural Number Game

Hilário Fernandes de Araujo Júnior
hilariofjunior@live.com

May 19, 2026

Contents

1	Tutorial World	2
1.1	The Natural Numbers	3
1.2	Addition	4
2	Addition World	5
3	Next Steps	6

Introduction

The objective of this document is to present my solutions to the exercises in the Natural Number Game in an informal style, to consolidate the knowledge acquired in the course. The theoretical content will be minimal, reflecting the style of the course.

The notation $a : A$ means that a is a term of type A . Terms are also called “inhabitants”. In type theory, each type comes equipped with rules that govern how its inhabitants are constructed and used. These rules come in two flavors:

- Introduction rules specify how to construct inhabitants of a type. They specify “what does it take to produce a term of the type”. The reflexivity rule refl_x is an example: it tells us that refl_x is a canonical inhabitant of $x = x$.
- Elimination rules specify how to use inhabitants of a type. They specify “what we can do with a term of a given type”. The rule elim is an example: it tells us that an inhabitant $h : x = y$ can be used to transport inhabitants between related types.

Given two types A and B , we can define the type of functions from A to B , denoted by $A \rightarrow B$. The function type $A \rightarrow B$ is governed by the following rules:

- $\rightarrow_{\text{intro}}$: the type $A \rightarrow B$ is inhabited if there is a way to assign to each inhabitant $a : A$ an inhabitant $b : B$. The canonical way to specify such an assignment is via a *lambda term* $\lambda a. t(a)$, where $t(a)$ is an expression of type B that may depend on a .
- $\rightarrow_{\text{elim}}$: given inhabitants $f : A \rightarrow B$ and $a : A$, there is an inhabitant $f(a) : B$.

Informally, we can think of a lambda term $\lambda a. t(a)$ as a function $a \mapsto t(a)$.

Given a type A and terms $x, y : A$, the identity type $x =_A y$ is governed by the following introduction and elimination rules:

- $=_{\text{intro}}$: given a type A and term $x : A$, there is a term $\text{refl}_x : x =_A x$.
- $=_{\text{elim}}$: given a type A , terms $x, y : A$, types $P(x)$ and $P(y)$ (we shall go over what $P(\cdot)$ means at a later time, but we can think informally of it as a family of types indexed by terms of another type) and $h : x = y$, then there exists an inhabitant $f(h) : P(x) \rightarrow P(y)$ induced by $h : x = y$.

Throughout this document, we will use the notation $x = y$ instead of $x =_A y$ when the type A is clear from the context.

In type theory, propositions are types and proofs are terms. Therefore, our objective is to provide witnesses to certain types, and that in turn proves our desired propositions. The reflexivity introduction rule means that propositions of the form $x =_A x$ are provable.

This elimination rule (here described informally) allows us to use identity (when it is inhabited) to “rewrite” terms, or “translate” inhabitants between types. Identity symmetry is a consequence of this rule:

Proposition 0.1. *Given $x, y : A$ and $h : x = y$, the type $y = x$ is inhabited.*

Proof. Consider the type family $P(z) := (z = x)$. Then $\text{refl}_x : P(x)$, i.e., $\text{refl}_x : x = x$, is a canonical inhabitant of $P(x)$. Applying the elimination rule for identity with $h : x = y$, we get an inhabitant of $P(y)$, i.e., of $y = x$. $\square$

The inhabitant constructed in the proof of Proposition 0.1 is called the “symmetric” of h , and it is denoted by h^{-1} .

One notational observation is worth mentioning: we write $:=$ for definitions, in accordance to Lean’s convention. Don’t confuse the colon in the notation $x : A$ with the colon in the notation $P(z) := (z = x)$. The first is a judgment; the second is a definition.

It is worth pausing to understand the difference between a *judgment* and a *proposition*. In classical set theory, the expression $a \in A$ is a *proposition*: it is a mathematical statement that is either true or false, and we can ask whether it is provable or refutable. In type theory, on the other hand, the expression $a : A$ is a *judgment*: it is a declaration made at the meta-level, asserting that the term a has been *constructed* in accordance with the formation rules of type A . A judgment is not something that can be true or false — it is either *valid* (accepted by the type-checker) or *invalid* (rejected). In other words, $a \in A$ is a claim we might need to prove, whereas $a : A$ is a syntactic fact about a term that is verified mechanically.

1 Tutorial World

We may begin by stating the first proposition we are asked to prove:

Proposition 1.1. *Given $x, y : \mathbb{N}$ and $h : y = x + 7$, the type $2 \cdot y = 2 \cdot (x + 7)$ is inhabited.*

Proof. We have $h : y = x + 7$ (thus $h^{-1} : x + 7 = y$). Let $P(z) := (2 \cdot z = 2 \cdot (x + 7))$. By the elimination rule for equality on h^{-1} , there exists an inhabitant $f(h^{-1}) : P(x + 7) \rightarrow P(y)$. We know that $P(x + 7)$ is inhabited by $\text{refl}_{2 \cdot (x + 7)}$, the canonical inhabitant given by the introduction rule for equality. Therefore, $P(y)$ is inhabited by $f(h^{-1})(\text{refl}_{2 \cdot (x + 7)})$. $\square$

Note here that the assumption $y = x + 7$ (which in logic is a proposition involving the binary relation $=$) here is described as the type $y = x + 7$ being inhabited by term h .

We could have proved $2 \cdot y = 2 \cdot (x + 7)$ by using h instead of h^{-1} :

Proof. Define the type family $Q(z) := (2 \cdot y = 2 \cdot z)$. By the elimination rule for equality on h , there exists an inhabitant $f(h) : Q(y) \rightarrow Q(x + 7)$. We know that $Q(y)$ is inhabited by $\text{refl}_{2 \cdot y}$, the canonical inhabitant given by the introduction rule for equality. Therefore, $Q(x + 7)$ is inhabited by $f(h)(\text{refl}_{2 \cdot y})$. $\square$

Note that these proofs are quite verbose, and would get even more verbose the more we apply the identity elimination rule (we would have to define a series of types $P_0(z), P_1(z)$, etc.). We can make it more concise by using the fact that the process of applying the elimination rules and “transporting” terms can be thought of as “rewriting types”: instead of trying to provide an inhabitant for $2 \cdot y = 2 \cdot (x + 7)$ we use the elimination rule to provide an inhabitant for $2 \cdot (x + 7) = 2 \cdot (x + 7)$ instead. An alternative, more informal proof would be:

Proof. By the assumption $h : y = x + 7$, we can rewrite the type $2 \cdot y = 2 \cdot (x + 7)$ as $2 \cdot (x + 7) = 2 \cdot (x + 7)$. The inhabitant $\text{refl}_{2 \cdot (x + 7)}$ concludes the proof. $\square$

This, in my view, provides a nice middle-ground between the verbose and the fully intuitive proof (which would be something like this: since $y = x + 7$, we can replace y by $(x + 7)$ in $2 \cdot y = 2 \cdot (x + 7)$ and have a tautology.).

1.1 The Natural Numbers

Now, we shall introduce the type of natural numbers, denoted by $\mathbb{N}$. This type has the following introduction rules:

- $\mathbb{N}_{\text{intro}}$: there is a term $0 : \mathbb{N}$
- $\mathbb{N}_{\text{intro}}$: there is a term $\text{succ} : \mathbb{N} \rightarrow \mathbb{N}$

This type also has an elimination rule (which generalizes the principle of induction), but we shall not need it for now. Now, let us define a few numbers:

$$\begin{aligned} 1 &:= \text{succ}(0) \\ 2 &:= \text{succ}(1) \\ 3 &:= \text{succ}(2) \\ 4 &:= \text{succ}(3) \end{aligned} \tag{1}$$

These definitions in particular mean that the types $1 = \text{succ}(0)$, $2 = \text{succ}(1)$, etc. are inhabited.

Proposition 1.2. *The type $2 = \text{succ}(\text{succ}(0))$ is inhabited.*

Proof. By the definition of 2, type $2 = \text{succ}(1)$ is inhabited. We can then rewrite the type $2 = \text{succ}(\text{succ}(0))$ as $\text{succ}(1) = \text{succ}(\text{succ}(0))$. By the definition of 1, we have $1 = \text{succ}(0)$. Thus, we can rewrite the type $\text{succ}(1) = \text{succ}(\text{succ}(0))$ as $\text{succ}(\text{succ}(0)) = \text{succ}(\text{succ}(0))$. The inhabitant $\text{refl}_{\text{succ}(\text{succ}(0))}$ concludes the proof. $\square$

The next proposition in the Natural Number Game is still 1.2, but the exercise is to provide the proof by rewriting the right-hand side of the equality. We’ve already done a similar thing for Proposition 1.1 (with some verbosity); the shorter-style proof for Proposition 1.2 is as follows:

Proof. By the definition of 1, type $1 = \text{succ}(0)$ is inhabited. We can then rewrite the type $2 = \text{succ}(\text{succ}(0))$ as $2 = \text{succ}(1)$. By the definition of 2, we have $2 = \text{succ}(1)$. Thus, we can rewrite the type $2 = \text{succ}(1)$ as $2 = 2$. The inhabitant refl_2 concludes the proof. $\square$

1.2 Addition

We now introduce the addition operation on natural numbers as a term $+$ of the function type $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$. A small note on this type signature is in order: in type theory, the function arrow is right-associative, meaning this is actually $\mathbb{N} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$. This reflects a technique called *currying*: instead of a single function taking two numbers, $+$ is a function that takes one number and returns a new function (the “add n ” function), which then takes the second number.

This syntactic convention takes the ambiguity out of the function signature itself, though it is important to distinguish this from the *associativity of the operator* $((a+b)+c = a+(b+c))$, which is a deep property that must be proven (by the way, Lean interprets the non-canonical $a+b+c$ as $(a+b)+c$). In alignment with the propositions-as-types paradigm, the properties of addition correspond to the inhabitancy of certain identity types. Addition is characterized by two fundamental such rules:

- **add_zero** : $+(n)(0) = n$, which means that for each $n : \mathbb{N}$, the type $+(n)(0) = n$ is inhabited. However, instead of writing $+(n)(0)$ we shall write $n + 0$. Thus, type $n + 0 = n$ is inhabited for all $n : \mathbb{N}$.
- **add_succ** : $+(n)(\text{succ}(m)) = \text{succ}(+(n)(m))$, which means that for each $n, m : \mathbb{N}$, the type $+(n)(\text{succ}(m)) = \text{succ}(+(n)(m))$ is inhabited. However, instead of writing $+(n)(\text{succ}(m))$ we shall write $n + (m + 1)$ and instead of writing $\text{succ}(+(n)(m))$ we shall write $\text{succ}(n + m)$. Thus, type $n + \text{succ}(m) = \text{succ}(n + m)$ is inhabited for all $n, m : \mathbb{N}$.

The first proposition we wish to prove is then:

Proposition 1.3. *The type $a + (b + 0) + (c + 0) = a + b + c$ is inhabited.*

Proof. By **add_zero**, $b + 0 = b$ and $c + 0 = c$ are inhabited. Thus, we can rewrite the type $a + (b + 0) + (c + 0) = a + b + c$ as $a + b + c = a + b + c$. The inhabitant refl_{a+b+c} concludes the proof. $\square$

The next proposition is:

Proposition 1.4. *The type $\text{succ}(n) = n + 1$ is inhabited.*

Proof. By the definition of 1, type $1 = \text{succ}(0)$ is inhabited. We can then rewrite the type $\text{succ}(n) = n + 1$ as $\text{succ}(n) = n + \text{succ}(0)$. By **add_succ**, type $n + \text{succ}(0) = \text{succ}(n + 0)$ is inhabited. Thus, we can rewrite the type $\text{succ}(n) = n + \text{succ}(0)$ as $\text{succ}(n) = \text{succ}(n + 0)$. By **add_zero**, type $n + 0 = n$ is inhabited. Thus, we can rewrite the type $\text{succ}(n) = \text{succ}(n + 0)$ as $\text{succ}(n) = \text{succ}(n)$. The inhabitant $\text{refl}_{\text{succ}(n)}$ concludes the proof. $\square$

The inhabitant of the type $\text{succ}(n) = n + 1$ will be called `succ_eq_add_one`. The next proposition uses this inhabitant and is the last one in this section:

Proposition 1.5. *The type $2 + 2 = 4$ is inhabited.*

Proof. By the definition of 4, type $4 = \text{succ}(3)$ is inhabited. We can then rewrite the type $2 + 2 = 4$ as $2 + 2 = \text{succ}(3)$. Similarly, by the definition of 3, type $3 = \text{succ}(2)$ is inhabited. Thus, we can rewrite the type $2 + 2 = \text{succ}(3)$ as $2 + 2 = \text{succ}(\text{succ}(2))$. By the definition of 2, type $2 = \text{succ}(1)$ is inhabited. Thus, we can rewrite the type $2 + 2 = \text{succ}(\text{succ}(2))$ as $2 + \text{succ}(1) = \text{succ}(\text{succ}(1))$. By `add_succ`, type $2 + \text{succ}(1) = \text{succ}(2 + 1)$ is inhabited. Thus, we can rewrite the type $2 + \text{succ}(1) = \text{succ}(\text{succ}(2))$ as $\text{succ}(2 + 1) = \text{succ}(\text{succ}(2))$. Now, by `succ_eq_add_one`, type $2 + 1 = \text{succ}(2)$ is inhabited. Thus, we can rewrite the type $\text{succ}(2 + 1) = \text{succ}(\text{succ}(2))$ as $\text{succ}(\text{succ}(2)) = \text{succ}(\text{succ}(2))$. The inhabitant $\text{refl}_{\text{succ}(\text{succ}(2))}$ concludes the proof. $\square$

2 Addition World

Now, we begin using the induction elimination rule for natural numbers. We shall still not define it formally (since it depends on dependent types and the universe hierarchy, which have not yet been introduced), but we can think of it as follows: given a type $P(n)$ that is indexed by $n \in \mathbb{N}$, the rule tells us that if $P(0)$ is inhabited, and if for all $n \in \mathbb{N}$, $P(n)$ being inhabited implies $P(\text{succ}(n))$ is inhabited, then $P(n)$ is inhabited for all $n \in \mathbb{N}$.

Our first proposition in this section is analogous to `add_zero` (rather, to the type $+(n)(0) = n$ which is inhabited by `add_zero`):

Proposition 2.1. *The type $0 + n = n$ is inhabited for all $n \in \mathbb{N}$.*

Proof. We can prove this proposition by induction on n . Base case: $n = 0$, meaning we wish to prove the type $0 + 0 = 0$ is inhabited. By `add_zero`, type $0 + 0 = 0$ is inhabited. Inductive step: Assume that the type $0 + n = n$ is inhabited for some $n \in \mathbb{N}$: we wish to show that type $0 + \text{succ}(n) = \text{succ}(n)$ is inhabited. By `add_succ`, type $0 + \text{succ}(n) = \text{succ}(0 + n)$ is inhabited. Thus, we can rewrite the type $0 + \text{succ}(n) = \text{succ}(n)$ as $\text{succ}(0 + n) = \text{succ}(n)$. Now, by the inductive hypothesis (the inhabitant of $0 + n = n$) we can rewrite the type $\text{succ}(0 + n) = \text{succ}(n)$ as $\text{succ}(n) = \text{succ}(n)$. The inhabitant $\text{refl}_{\text{succ}(n)}$ concludes the proof. $\square$

The inhabitant will be called `zero_add`.

Now, we wish to prove the following:

Proposition 2.2. *The type $\text{succ}(a) + b = \text{succ}(a + b)$ is inhabited for all $a, b \in \mathbb{N}$.*

Proof. We can prove this proposition by induction on b . Base case: $b = 0$, meaning we wish to prove $\text{succ}(a) + 0 = \text{succ}(a + 0)$ is inhabited. By `add_zero`, type $\text{succ}(a) + 0 = \text{succ}(a)$ is inhabited. Thus, we can rewrite the type $\text{succ}(a) + 0 = \text{succ}(a + 0)$ as $\text{succ}(a) = \text{succ}(a + 0)$. By `add_zero` again, type $a + 0 = a$ is inhabited. Thus, we can rewrite the type $\text{succ}(a) = \text{succ}(a + 0)$ as $\text{succ}(a) = \text{succ}(a)$. The inhabitant $\text{refl}_{\text{succ}(a)}$ concludes the proof of the base case.

Inductive step: Assume that the type $\text{succ}(a) + b = \text{succ}(a + b)$ is inhabited for some $b \in \mathbb{N}$. We wish to show that type $\text{succ}(a) + \text{succ}(b) = \text{succ}(a + \text{succ}(b))$ is inhabited. By `add_succ`, type $\text{succ}(a) + \text{succ}(b) = \text{succ}(\text{succ}(a) + b)$ is inhabited. Thus, we can rewrite

the type $\text{succ}(a) + \text{succ}(b) = \text{succ}(a + \text{succ}(b))$ as $\text{succ}(\text{succ}(a) + b) = \text{succ}(a + \text{succ}(b))$. By `add_succ` again, type $\text{succ}(\text{succ}(a) + b) = \text{succ}(\text{succ}(a + b))$ is inhabited. Thus, we can rewrite the type $\text{succ}(\text{succ}(a) + b) = \text{succ}(a + \text{succ}(b))$ as $\text{succ}(\text{succ}(a) + b) = \text{succ}(\text{succ}(a + b))$. Now, by the inductive hypothesis (the inhabitant of $\text{succ}(a) + b = \text{succ}(a + b)$) we can rewrite the type $\text{succ}(\text{succ}(a) + b) = \text{succ}(\text{succ}(a + b))$ as $\text{succ}(\text{succ}(a + b)) = \text{succ}(\text{succ}(a + b))$. The inhabitant $\text{refl}_{\text{succ}(\text{succ}(a+b))}$ concludes the proof. $\square$

Finally, we're able to prove the commutativity of addition:

Proposition 2.3. *The type $a + b = b + a$ is inhabited for all $a, b \in \mathbb{N}$.*

Proof. We shall prove this by induction on b . Base case: $b = 0$, meaning we wish to prove $a + 0 = 0 + a$ is inhabited. By `zero_add`, type $0 + a = a$ is inhabited. Thus, we can rewrite the type $a + 0 = 0 + a$ as $a + 0 = a$. But by `add_zero`, type $a + 0 = a$ is inhabited. Thus, we can rewrite $a = a + 0$ as $a = a$. The inhabitant refl_a concludes the proof of the base case.

Inductive step: Assume that the type $a + d = d + a$ is inhabited for some $d \in \mathbb{N}$. We wish to show that type $a + \text{succ}(d) = \text{succ}(d) + a$ is inhabited. By `add_succ`, type $a + \text{succ}(d) = \text{succ}(a + d)$ is inhabited. Thus, we can rewrite the type $a + \text{succ}(d) = \text{succ}(d) + a$ as $\text{succ}(a + d) = \text{succ}(d) + a$. Now, by `succ_add`, type $\text{succ}(d) + a = \text{succ}(d + a)$ is inhabited. Thus, we can rewrite $\text{succ}(a + d) = \text{succ}(d) + a$ as $\text{succ}(a + d) = \text{succ}(d + a)$. Now, by the inductive hypothesis (the inhabitant of $a + d = d + a$), we can rewrite the type $\text{succ}(a + d) = \text{succ}(d + a)$ as $\text{succ}(d + a) = \text{succ}(d + a)$. The inhabitant $\text{refl}_{\text{succ}(d+a)}$ concludes the proof. $\square$

After commutativity, we prove associativity:

Proposition 2.4. *The type $(a + b) + c = a + (b + c)$ is inhabited for all $a, b, c \in \mathbb{N}$.*

Proof. We shall prove this by induction on c . Base case: $c = 0$, meaning we wish to prove $(a + b) + 0 = a + (b + 0)$ is inhabited. By `add_zero`, type $b + 0 = b$ is inhabited. Thus, we can rewrite the type $(a + b) + 0 = a + (b + 0)$ as $(a + b) + 0 = a + b$. But by `add_zero` again, type $(a + b) + 0 = a + b$ is inhabited. Thus, we can rewrite $(a + b) + 0 = a + b$ as $a + b = a + b$. The inhabitant refl_{a+b} concludes the proof of the base case.

Inductive step: Assume that the type $(a + b) + d = a + (b + d)$ is inhabited for some $d \in \mathbb{N}$. We wish to show that type $(a + b) + \text{succ}(d) = a + (b + \text{succ}(d))$ is inhabited. By `add_succ`, type $(a + b) + \text{succ}(d) = \text{succ}((a + b) + d)$ is inhabited. Thus, we can rewrite the type $(a + b) + \text{succ}(d) = a + (b + \text{succ}(d))$ as $\text{succ}((a + b) + d) = a + (b + \text{succ}(d))$. Now, by `add_succ` again, type $b + \text{succ}(d) = \text{succ}(b + d)$ is inhabited. Thus, we can rewrite $\text{succ}((a + b) + d) = a + (b + \text{succ}(d))$ as $\text{succ}((a + b) + d) = a + \text{succ}(b + d)$. Now, by `add_succ` again, type $a + \text{succ}(b + d) = \text{succ}(a + (b + d))$ is inhabited. Thus, we can rewrite $\text{succ}((a + b) + d) = a + \text{succ}(b + d)$ as $\text{succ}((a + b) + d) = \text{succ}(a + (b + d))$. Now, by the inductive hypothesis (the inhabitant of $(a + b) + d = a + (b + d)$), we can rewrite the type $\text{succ}((a + b) + d) = \text{succ}(a + (b + d))$ as $\text{succ}(a + (b + d)) = \text{succ}(a + (b + d))$. The inhabitant $\text{refl}_{\text{succ}(a+(b+d))}$ concludes the proof. $\square$

3 Next Steps

Mention further topics for study, such as lambda calculus, dependent types, universe hierarchy, etc. as a sort of "appendix".